CHỦ ĐỀ 2 · MÔN LẬP TRÌNH ĐA NỀN TẢNG

Lập trình bất đồng bộ với Future & Stream

Ứng dụng demo: Multi-Stopwatch, đồng hồ bấm giờ đa tác vụ

Báo cáo	Tuần 1
Thành viên	Đặng Công Thắng (MSSV: 106230061)
	Võ Văn Thái (MSSV: 1062300)
Giảng viên	Nguyễn Duy Nhật Viễn

Trường Đại học Bách khoa – Đại học Đà Nẵng · Khoa Điện tử Viễn thông

Mã nguồn công khai: `github.com/cityengin/multi-stopwatch`

Phân công

Đặng Công Thắng · Business Logic & Báo cáo

- Business Logic: Stream, service (`lib/core`)
- Database/API giả lập: repository, độ trễ, retry
- Báo cáo và slide

Võ Văn Thái · UI & State Management

- UI Design: các màn hình của app
- State Management: `StreamBuilder`, `InheritedWidget`
- Widget test, hủy subscription ở tầng UI

Tỷ lệ đóng góp

Đặng Công Thắng 55%	Võ Văn Thái 45%

Lịch sử commit công khai theo từng thành viên: `github.com/cityengin/multi-stopwatch`

Tổng quan dự án

Multi-Stopwatch

Nhiều đồng hồ chạy song song, độc lập, hiển thị thời gian thực.

- Start, Pause, Resume, Reset, Lap
- Chạy, dừng, xóa hàng loạt
- Lưu cục bộ, khôi phục sau khi mở lại
- Tìm kiếm, màn Debug, Event Loop Lab

Yêu cầu của chủ đề

Nội dung

- Event Loop, Microtask Queue
- Future, async/await
- Stream, StreamController, Broadcast Stream
- Toán tử biến đổi dữ liệu: Transform, Debounce

Yêu cầu demo

Ứng dụng theo dõi dữ liệu thời gian thực (đồng hồ bấm giờ đa tác vụ đa luồng), có xử lý **hủy luồng (cancellation/dispose) an toàn** khi thoát màn hình.

Yêu cầu và công nghệ

Phần mềm

- Flutter 3.47.7 (stable)
- Dart 3.13.5
- Android Studio, SDK, JDK 17+
- Git, GitHub

Thư viện

- shared_preferences lưu cục bộ
- meta (`@immutable`)
- fake_async thời gian ảo, khi test
- flutter_test, flutter_lints
- cupertino_icons, flutter_launcher_icons

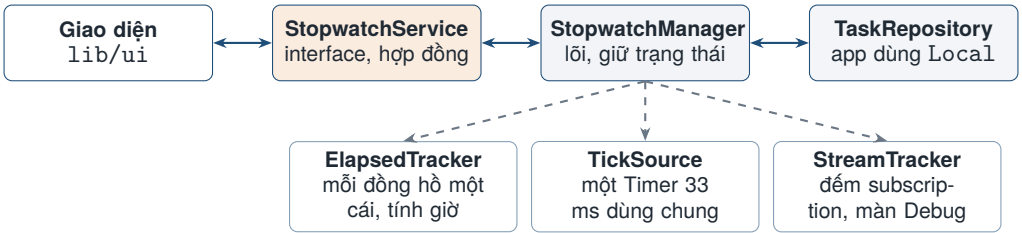
Chạy thử

- Máy thật: Pixel 8, Android 17.0
- `flutter pub get`
- `flutter run`

Không cần cấu hình thêm

Không API key, không `.env`, không Firebase. Dữ liệu cục bộ hoặc giả lập trong bộ nhớ; không rxdart, không thư viện state management ngoài.

Kiến trúc



UI chỉ biết `StopwatchService`: đổi cách lưu hay tính giờ, UI không phải sửa

Một `StopwatchManager` cho cả app; mỗi đồng hồ có `ElapsedTracker` riêng

Một `TickSource` dùng chung, chỉ chạy khi có người xem

Dùng gì. Flutter và Dart thuần (`Timer`, `Stream`, `Future`), `shared_preferences` để lưu. Không Bloc, Provider, rxdart.

Xử lý một lệnh: xếp hàng, cập nhật, lưu, hoàn tác

`start(id)` → `_enqueue` → `_mutate` → báo UI **ngay** → lưu → lỗi: hoàn tác

1. `_enqueue`: hàng đợi lệnh

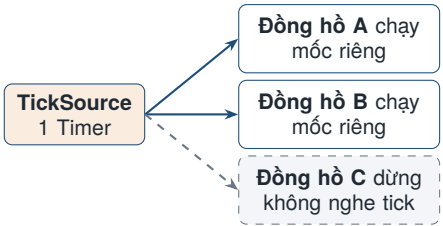
```
Future<T> _enqueue<T>(
    Future<T> Function() command) {
  // Đã dispose thì từ chối lệnh mới
  if (_isDisposed) {
    return Future.error(StateError('disposed'));
  }
  // Nối lệnh vào đuôi hàng: chạy sau khi lệnh trước xong
  final result =
      _queueTail.then((_) => command());
  // Đuôi nuốt lỗi: không thì một lệnh lỗi chặn mọi lệnh sau
  _queueTail = result.then<void>((_) {},
      onError: (Object _) {});
  return result;  // người gọi vẫn nhận lỗi
}
```

2. `_mutate`: đổi, báo UI, lưu

```
_enqueue(() async {
  final entry = _entryOf(taskId);
  final previous = entry.task;
  apply(entry);   // idle -> running
  _emitTasks();   // báo UI trước
  try {
    await _repository.upsert(entry.task);
  } catch (_) {
    entry.restore(previous);
    // lưu lỗi: hoàn tác, báo UI lại
    _emitTasks();
    rethrow;
  }
});
```

-> **Future, async/await**: mỗi lệnh là một `Future` nối vào đuôi hàng đợi; `await` chờ lưu mà không chặn UI; lỗi đi qua `Future` nên bắt bằng `try/catch`.

TickSource




```dart
// TickSource, abridged
StreamController<int>.broadcast(
  onListen: _startTimer,   // 0 -> 1 listener
  onCancel: _stopTimer,    // 1 -> 0 listener
);
void _startTimer() {
  _timer ??= Timer.periodic(interval,
      (t) => _controller.add(t.tick));
}
// _TaskEntry, abridged: listen if running
if (tracker.isRunning && hasListeners) {
  _tickSub ??= ticks.listen((_) => _emit());
} else { _tickSub?.cancel(); }
```

- **Một** Timer cho N đồng hồ
- Mỗi đồng hồ giữ mốc giờ **riêng**
- Tick đến: đồng hồ đang chạy tự tính lại giờ

-> **Event Loop**: một luồng, mỗi tick là một event; các đồng hồ xen kẽ nhau trên cùng luồng. **Broadcast**: một nguồn tick, nhiều người nghe; `onListen`/`onCancel` bật tắt Timer.

ElapsedTracker: tính giờ theo mốc

Sai: cộng dồn tick

`elapsed += 33 ms` mỗi tick. Timer chạy trên event loop nên trễ hoặc bỏ nhịp khi luồng bận ⇒ sai số chỉ tích lũy.

Đúng: tính theo mốc

```
elapsed = accumulated
   + (now - runningSince)
```

Tick chỉ là lúc đọc lại: trễ thì đọc trễ, giá trị vẫn đúng.

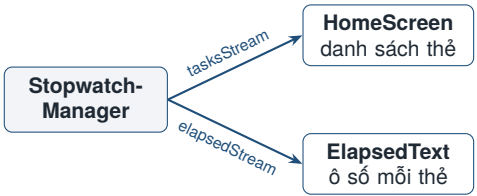
```
Duration get elapsed {
  final segment = _syncSegment();
  return segment == null
      ? _accumulated
      : _accumulated + segment;
}
```

Hai nguồn thời gian (`TimeSource`)

Stopwatch (đơn điệu): không nhảy khi chỉnh giờ.
DateTime.now: lưu mốc, còn sau khi tắt app.
Đối chiếu hai nguồn để bù lúc máy ngủ.

Kết quả test (fake_async). 1 giờ ảo, cứ 90 tick chặn event loop 400 ms: tính theo mốc lệch **0 ms**, cộng dồn tick hụt **hơn 5 phút**.

Dữ liệu tới màn hình: hai luồng




```dart
// elapsedStream(taskId), abridged
Stream.multi((controller) {
  entry.addListener(controller);
}, isBroadcast: true);
// _TaskEntry.addListener
final sink = _LatestValueSink(controller);
_elapsedSinks.add(sink);
sink.emit(tracker.elapsed);
controller.onCancel = () {
  _elapsedSinks.remove(sink);
  _syncTicking();
};
```

- `tasksStream`: phát khi trạng thái đổi, **không** theo tick
- `elapsedStream(id)`: mỗi đồng hồ một luồng, theo tick khi chạy
- Listener mới nhận ngay giá trị hiện tại

-> **Stream**: chuỗi giá trị theo thời gian (`Future` chỉ một giá trị). **Broadcast**: nhiều người nghe cùng lúc, single-subscription chỉ một; cả hai luồng này là broadcast.

Throttle và debounce: giảm tần suất cập nhật



mỗi vạch = 100 ms, duration = 300 ms

- **Throttle**: phát ngay, rồi bỏ qua một khoảng. Dự án: 33 ms ở `ElapsedText`, giảm số lần vẽ.
- **Debounce**: chờ im lặng đủ lâu mới phát. Dự án: 300 ms ở ô tìm kiếm.

```dart
// _ThrottleGate.onData
if (_window == null) {
  sink.add(event);
  _openWindow(sink);
} else if (_trailing) {
  _hasTrailing = true;
  _trailingEvent = event;
}
```

```dart
// _DebounceGate.onData
_timer?.cancel();
_hasPending = true;
_pending = event;
_timer = Timer(_duration,
    () => _flush(sink));
```

`rate_limit.dart`: tự viết bằng `StreamTransformer`, không dùng rxdart.

Lưu trữ và khôi phục

- `LocalTaskRepository`: shared_preferences, một key JSON có `version`
- Một lần ghi là một `setString`: không ghi nửa chừng; lệnh hàng loạt ghi **một** lần
- Mở lại app: `ElapsedTracker` dựng lại từ `runningSince`, đồng hồ chạy tính cả lúc app tắt
- `InMemory` (giả lập trễ, lỗi) và `Retrying` (thử lại) có test; app chạy bản Local

```dart
// LocalTaskRepository._write, abridged
final raw = jsonEncode({
  'version': schemaVersion,
  'tasks': [for (final t in ts) t.toJson()],
});
final saved = await _preferences
    .setString(key, raw);
if (!saved) throw StorageException('failed');

// ElapsedTracker: restore on launch
final gap = wallNow.difference(since);
_runningSince = wallNow.subtract(gap);
_monotonicAnchor =
    _time.monotonicNow() - gap;
```

-> **Future**: lưu là I/O bất đồng bộ nên trả `Future`; lỗi đi qua `Future` thành `StorageException`, `_mutate` bắt để hoàn tác.

Hủy luồng an toàn (dispose)

```dart
Future<void> _dispose() async {  // abridged
  await _queueTail;       // wait pending
  for (final entry in _entries.values) {
    entry.close();        // cancel + done
  }
  for (final sink in [..._taskSinks]) {
    sink.close();
  }
  if (_ownsTickSource) {
    await _tickSource.dispose();
  }
  _tracker?.recordDispose('StopwatchManager');
}
```

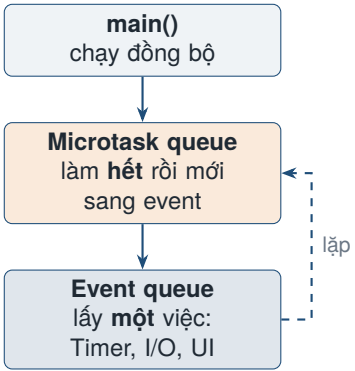
- Gọi nhiều lần không lỗi (idempotent)
- Hủy mọi subscription và Timer, đóng controller; hết người nghe → Timer tự dừng

Log của StreamTracker

```
listen elapsedStream
listen TickSource.ticks
cancel TickSource.ticks
done elapsedStream
dispose TickSource
dispose StopwatchManager
```

-> **Hủy luồng**: `StreamSubscription.cancel()` và đóng `StreamController` đúng lúc; không hủy thì Timer và listener vẫn sống (rò rỉ).

Event Loop Lab (1): thứ tự sync, microtask, event

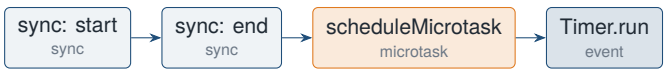


Từ dự án: một kịch bản của Event Loop Lab

```
void _microtaskVsTimer(LabLog log) {
  log('sync: start');
  Timer.run(() => log('Timer.run'));
  scheduleMicrotask(
      () => log('scheduleMicrotask'));
  log('sync: end');
}
```

`event_lab_runner.dart`. Dart chạy một luồng; `Timer.run` đăng ký trước nhưng microtask vẫn chạy trước, vì microtask luôn được dọn sạch trước khi lấy việc từ event queue.

Thứ tự chạy thật



Event Loop Lab (2): đo thứ tự bằng Zone

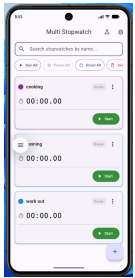
```
final zone = Zone.current.fork(
  specification: ZoneSpecification(
    scheduleMicrotask: (self, parent, zone, cb) {
      pending++;
      parent.scheduleMicrotask(zone,
          () => runFrom(LabQueue.microtask, cb));
    },
    createTimer: (self, parent, zone, d, cb) {
      pending++;
      return parent.createTimer(zone, d,
          () => runFrom(LabQueue.event, cb));
    },
  ),
);
zone.run(() => scenario._body(log));
```

Rút gọn tên tham số từ `EventLabRunner.run`.

`EventLabRunner` chạy 6 kịch bản trong một **Zone** riêng:

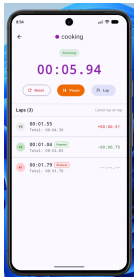
- Zone chặn `scheduleMicrotask` và `createTimer`
- Mỗi callback được gắn đúng hàng đợi mà nó **thực sự** đến từ đó
- `pending` đếm việc đang chờ: biết chắc khi nào kịch bản xong
- Có test đối chiếu thứ tự với lý thuyết

Các màn hình của ứng dụng



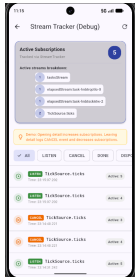
Màn chính

Danh sách, tìm kiếm, thao tác hàng loạt



Chi tiết

Số lớn, nút điều khiển, bảng lap



Debug

Đếm subscription đang sống

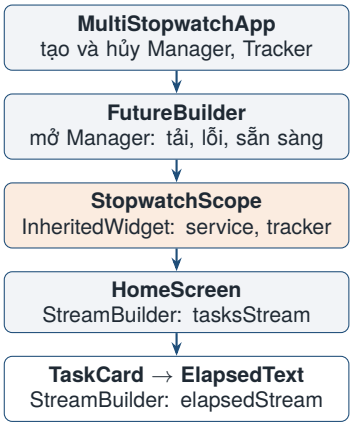


Event Loop Lab

Thứ tự sync, microtask, event

Từ màn chính: chạm một thẻ để mở màn chi tiết; hai biểu tượng trên thanh tiêu đề mở Debug và Event Loop Lab.

Quản lý trạng thái: Stream, không cần thư viện ngoài

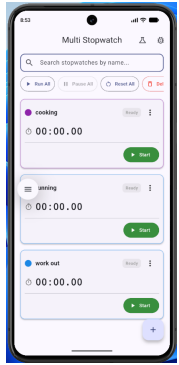


State = **Stream** + `StatefulWidget` + `InheritedWidget`

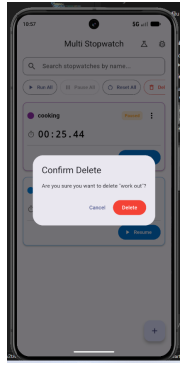
- **Một** `StopwatchManager`: tạo ở `initState`, hủy ở `dispose` của widget gốc
- `StopwatchScope` đưa service và tracker xuống mọi widget con
- Stream lấy một lần ở `initState`, không tạo trong `build()`
- `ElapsedText`: mỗi tick chỉ vẽ lại ô số, không vẽ lại thẻ hay danh sách

Màn chính: thao tác hàng loạt và tìm kiếm

- Thanh công cụ: **Run All, Pause All, Reset All, Delete All**
- Nút tự tắt khi không có việc để làm (vd Pause All khi không đồng hồ nào chạy)
- Reset All và Delete All có **hộp thoại xác nhận**
- Tìm kiếm theo tên, **debounce 300 ms** lấy từ core
- Lưu lỗi: SnackBar đỏ và trạng thái được hoàn tác



Thanh hàng loạt và tìm kiếm



Xác nhận xóa

Thẻ đồng hồ và bố cục thích ứng

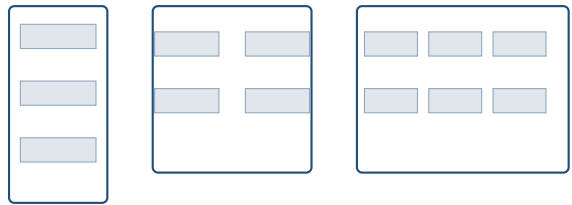
Thẻ đồng hồ (TaskCard)

- Tên, màu, trạng thái và số đo bằng `ElapsedText`
- Nút Start, Pause, Resume ngay trên thẻ
- Menu: đổi tên, đổi màu (8 màu), xóa

Tạo đồng hồ

- Nút **+**: nhập tên, chọn màu
- Danh sách rỗng hoặc không có kết quả tìm kiếm: có màn thông báo riêng

Bố cục theo bề rộng (`LayoutBuilder`)

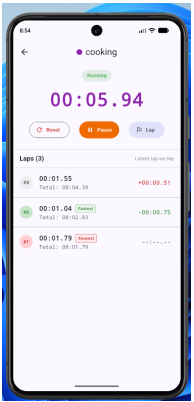


< 600 px	≥ 600 px	≥ 960 px
ListView	**Grid 2 cột**	**Grid 3 cột**

Có widget test: 400 px hiển thị danh sách, 800 px hiển thị lưới.

Màn chi tiết và Lap

- Số lớn dạng `mm:ss.cs` do `ElapsedText` vẽ
- Nút Start, Pause, Resume, Reset và **Lap** (ghi một vòng)
- Bảng vòng: **split** (thời gian vòng), **total**, **delta** so với vòng trước
- Vòng nhanh nhất và chậm nhất được đánh dấu (từ 2 vòng)
- Vòng mới nhất nằm trên cùng
- Đồng hồ bị xóa khi đang xem: hiện thông báo và nút quay lại



Màn chi tiết với bảng lap

Code giao diện: throttle, debounce, dispose

ElapsedText: giảm số lần vẽ

```
void _initStream(StopwatchService s) {
  var stream = s.elapsedStream(taskId);
  if (throttleDuration != null) {
    stream = stream.throttled(
      throttleDuration!,
      trailing: true);
  }
  _elapsedStream = stream;
}
```

`trailing: true` để không mất giá trị cuối khi Pause hoặc Reset. Cả hai đoạn code đã rút gọn từ mã nguồn.

HomeScreen: tìm kiếm và dispose

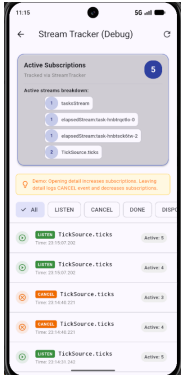
```
_searchSubscription = _searchController
    .stream
    .debounced(const Duration(
        milliseconds: 300))
    .listen((query) {
  if (mounted) setState(() =>
      _searchQuery = query);
});

void dispose() {
  _searchSubscription?.cancel();
  _searchController.close();
  super.dispose();
}
```

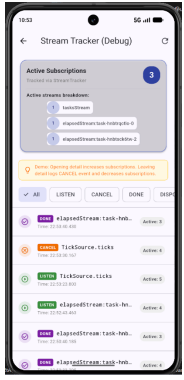
Demo: màn Debug chứng minh hủy luồng

1. Mở vài đồng hồ, bấm Chạy tất cả
2. Màn Debug: số subscription đang hoạt động tăng
3. Thoát màn hình hoặc xóa đồng hồ
4. Số subscription **giảm đúng**, log có `cancel`

Không hủy thì số này tăng dần sau mỗi lần vào ra: đó là rò rỉ.



Đang chạy



Sau khi thoát

Kết quả kiểm thử

flutter test

221 test All tests passed!

flutter analyze

0 vấn đề No issues found!

Chứng minh	Test (phần core)
Sau dispose: không listener, Timer, subscription	`stopwatch_manager_test`
Giờ không trôi khi chạy lâu	`elapsed_tracker_test`
Broadcast bật/tắt Timer theo listener	`tick_source_test`
Throttle, debounce đúng thời gian	`rate_limit_test`
Event Loop khớp lý thuyết	`event_lab_runner_test`

Kiểm thử giao diện

Widget test với `FakeStopwatchService` viết tay (không dùng mocktail); nằm trong **221 test** ở slide trước.

Chạy thật

Pixel 8, Android 17.0
`flutter analyze`: No issues found!

Test	Chứng minh
`subscription_lifecycle_test`	rời màn hình thì subscription được hủy
`home_screen_test`	thanh hàng loạt, tìm kiếm, hộp xác nhận
`elapsed_text_test`	chỉ ô số bị vẽ lại
`debug_screen_test`	màn Debug hiển thị đúng
`task_detail_screen_test`	màn chi tiết, lap
`event_lab_screen_test`	màn Event Loop Lab
`app_smoke_test`	app khởi động end-to-end

Tài liệu tham khảo I

- Eric Windmill (2020), *Flutter in Action*, Manning Publications.
- John Doe (2023), *A Comparative Study on State Management in Flutter*, IEEE Access, vol. 11, pp. 1200–1210.
- Flutter Documentation, *Flutter architectural overview*, `https://docs.flutter.dev`.
- Dart Language, *Dart programming language documentation*, `https://dart.dev`.